

Grupa: Numele și prenumele (cu MAJUSCULE):

Arhitectura Sistemelor de Calcul: Test - Laboratorul 0x04, 10% din nota finală

Exercițiul 1 (1.5p) Scrieți dimensiunea alocată în RAM, atât în biți, cât și în Bytes, pentru următoarele variabile declarate în secțiunea `.data`:

Declarare în <code>.data</code>	dimensiune în biți	dimensiune în Bytes
<code>str1: .asciz "Test ASC L0x04\n"</code>		
<code>str2: .asciz "Test ASC L0x04\n"</code>		
<code>myWord: .word 255</code>		
<code>myLong: .long 2703</code>		
<code>myByte: .byte 'a'</code>		

Este o declarare în secțiunea `.data` de forma `x: y: .long 15` validă? Argumentați răspunsul.

Scrieți un apel de sistem `WRITE` (codul apelului este 4) pentru șirul de caractere `str2`, astfel încât să afișați, la standard output, întreg conținutul șirului de caractere și nimic altceva în plus.

Exercițiul 2 (0.5p) Completați următorul tabel, cu valorile corecte în bazele 2, 8 și 16.

Baza 2	Baza 8	Baza 16
		41 20 00 00

Exercițiul 3 (1p) Fie următorul program, scris în *assembly* (procesor Intel x86, sintaxa AT&T):

```
.data                                mull x
    x: .long 0x41200000             et_exit:
.text                                movl $1, %eax
.global main                        xorl %ebx, %ebx
main:                               int $0x80
    movl $8, %eax
```

Executăm, în debugger, următoarele comenzi:

`b main; b et_exit; run; c; i r edx`

Valoarea pe care o vom obține în registrul EDX este

Exercițiul 4 (1p) Considerăm că, în programul de la exercițiul anterior, după `mull x` inserăm instrucțiunile

```
movl $2, %ebx
div %ebx
```

(restul programului rămâne identic, deci după aceste două instrucțiuni urmează `et_exit`)

Executăm, în debugger, următoarele comenzi:

`b et_exit; run`

Descrieți ce ar trebui să se întâmple în acest punct, utilizând modul în care se execută instrucțiunea `div`:

Ce se întâmplă dacă modificăm atribuirea în `EBX` să fie `movl $16, %ebx`? (înainte de a executa `div %ebx`)

Exercițiul 5 (1p) Fie următorul program, scris în *assembly* (procesor Intel x86, sintaxa AT&T):

```
.data                                xorl %edx, %edx                                movl $1, %ebx
    x: .long 18                      divl y                                jmp et_exit
    y: .long 20                      addl %edx, %eax                        et2:
.text                                mul %eax                                movl $2, %ebx
.global main                         sub %edx, %eax                        et_exit:
main:                                decl %eax                               movl $1, %eax
    movl x, %eax                     cmp $272, %eax                         xorl %ebx, %ebx
    addl y, %eax                     je et2                                int $0x80
    subl $2, %eax                    et1:
```

Completați valorile regiștrilor în momentul în care executarea programului ajunge în dreptul etichetei `et_exit` (după ce am rulat în debugger comenzile `b et_exit; run; i r`):

EAX	EBX	EDX

Exercițiul 6 (1p) Fie următorul program, dezvoltat în limbajul de asamblare al procesorului Intel x86, sintaxa AT&T:

```
.data                                movw %ax, %cx                                loop et_while
.text                                movb %ch, %cl                        et_exit:
.global main                         xorb %ch, %ch                         movl $1, %eax
main:                                movb %ah, %bl                         xorl %ebx, %ebx
    movl $0xa5cf05c0, %eax           movb %al, %bh                         int $0x80
    xorl %ecx, %ecx                  et_while:
    xorl %ebx, %ebx                  addb %bl, %bh
```

Valoarea stocată în dreptul etichetei `et_exit` în registrul `EBX` (în baza 16) este:

Exercițiul 7 (1p) Explicați dacă următoarele două secvențe de cod accesează elementul de pe indexul al doilea al unui array de întregi, stocat în RAM la adresa cu numele `v`. Sunt secvențele de cod echivalente? Dacă da, de ce? Dacă nu, ce accesează fiecare, în parte?

- a. `mov $v, %edi; movl $2, %ecx; movl (%edi, %ecx, 4), %ebx`
 b. `mov $v, %edi; addl $8, %edi; movl $0, %ecx; movl (%edi, %ecx, 4), %ebx`

Exercițiul 8 (2p) Scrieți o secvență de cod în `main` care să calculeze suma pătratelor elementelor dintr-un array. Considerați că lungimea array-ului este `n`, iar numele lui este `v`. Aceste informații sunt deja în secțiunea `.data`. Soluția voastră trebuie să înceapă de la eticheta `main`.